

INF 1036

Probabilidade Computacional

Material Extra – Revisão Python



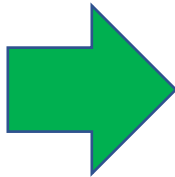
Funções

- A palavra reservada **def** permite definir funções.

```
def nome_da_função (parametro1, parametro2, etc...):  
    corpo_da_função
```

```
def calculaY(x):  
    retorno = x**2 + 3  
    return(retorno)
```

```
print(calculaY(5))
```



28

A instrução **return** permite que uma função retorne um valor.

Para chamar uma função, deve ser usado o nome da função seguido de parênteses.

Por padrão, se a função espera 2 **parâmetros**, ela deve ser chamada com 2 **argumentos**, nem mais nem menos.

Os parâmetros podem ser de quaisquer tipo de dados, sendo tratados como do mesmo tipo dentro da função.

Funções

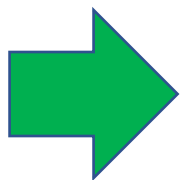
- Em Python é possível enviar argumentos usando a sintaxe **chave = valor**.

A função `str` converte o número inteiro `idade` em string para permitir a concatenação de strings feita com o operador `+`.

```
def imprimePessoa(nome, idade):  
    print('O nome da pessoa é ' + nome + ' e sua idade é ' + str(idade) + ' anos.')
```



```
imprimePessoa(idade = 30, nome = 'Pedra da Silva')
```



O nome da pessoa é Pedra da Silva e sua idade é 30 anos.

Os parâmetros foram passados via **chave = valor**. Observe que, neste caso, a ordem dos parâmetros pode não ser a mesma da definição da função.

Funções

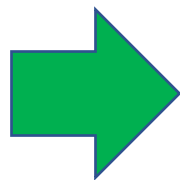
- Objetos **imutáveis** são passados por valor, criando uma nova cópia dentro do escopo da função.
- Ao passar objetos **mutáveis** para funções em Python, as alterações feitas dentro da função são refletidas no objeto original fora da função.

```
def processarValores (valorX, valorY):  
    valorX = valorX + 1  
    valorY.append(4)
```

```
x = 1  
y = [1, 2, 3]
```

```
processarValores(x, y)
```

```
print ('x : ', x)  
print ('y : ', y)
```



```
x : 1  
y : [1, 2, 3, 4]
```

Mutáveis: listas, dicionários, arrays NumPy e a maioria dos tipos definidos pelos usuários.

Imutáveis: tipos numéricos, como int, float e complex, strings, tuplas e tipo boolean.

Funções

- Tratamento de **valor padrão** de **parâmetro de função** em Python.

Parâmetros que possuem **valor padrão** devem ficar após os parâmetros que não possuem valor padrão.

```
def calculaHorasTrabalhadas(dias, jornada = 8):  
    return (dias * jornada)
```

```
horas = calculaHorasTrabalhadas(10)
```

O parâmetro **dias** irá receber o valor **10** e o parâmetro **jornada**, não obrigatório, irá assumir o valor **8**, neste exemplo. A variável **horas** irá receber o retorno da função, neste exemplo, o valor **80**.

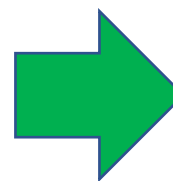
Funções

- Em Python, adicionar um `*` antes do nome de um parâmetro na definição da função irá permitir que ele receba uma **tupla** como argumento.

```
def imprimeSorteio(texto, *numeros):  
    print(texto)  
    for numero in numeros:  
        print(numero)
```

```
imprimeSorteio('Números sorteados: ', 30, 42, 19)
```

O parâmetro **numeros** possui o modificador `*` na sua definição permitindo que ele receba uma **tupla**.



Números sorteados:
30
42
19

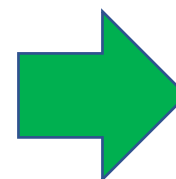
As **tuplas** no Python são tipos de dados semelhantes ao tipo **list**, porém, imutáveis (seus valores não podem ser modificados). No exemplo, o parâmetro **numero** será uma **tupla** com os valores **30**, **42** e **19** passados na chamada da função.

Funções

- Em Python, adicionar ****** antes do nome de um parâmetro na definição da função irá permitir que ele receba um **dicionário** como argumento.

```
def imprimeSorteio(texto, **numeros):  
    print(texto)  
    for chave in numeros:  
        print(chave, ' = ', numeros[chave])
```

O parâmetro **numeros** possui o modificador ****** na sua definição permitindo que ele receba um **dicionário**.



```
imprimeSorteio('Números sorteados: ', Primeiro = 30, Segundo = 42, Terceiro = 19)
```

Números sorteados:
Primeiro = 30
Segundo = 42
Terceiro = 19

Os **dicionários** no Python são tipos de dados usados para armazenar dados em pares **chave : valor**.
No exemplo, **numeros** será um dicionário sendo **Primeiro**, **Segundo** e **Terceiro** as chaves para os valores **30**, **42** e **19**, respectivamente.

Expressões regulares

- As expressões regulares possuem uma **sintaxe própria** e alguns dos metacaracteres (caracteres especiais) mais comuns são:
 - . (ponto) - representa qualquer caractere, exceto uma nova linha.
 - * (asterisco) - representa zero ou mais ocorrências do caractere ou grupo anterior.
 - + (mais) - representa uma ou mais ocorrências do caractere ou grupo anterior.
 - ? (opcional) - representa zero ou uma ocorrência do caractere ou grupo anterior.
 - ^ (circunflexo) - verifica o início da linha.
 - \$ (cifrão) - verifica o final da linha.
 - \ (escape) - utilizado para escapar caracteres especiais.
 - [] (lista) - representa um conjunto de caracteres.
 - [^] (lista negada) - representa um conjunto de caracteres proibidos.
 - () (grupo) - agrupa uma sequência de caracteres.
 - {**n**, **m**} (chaves) - encontra pelo menos *n* e no máximo *m* ocorrências da expressão precedente.

Exemplo

```
import re

resultado = re.match('Av', 'Av Santos Dumont')
print (resultado)
print (resultado.group(0))
print (resultado.start())
print (resultado.end())
```

```
<re.Match object; span=(0, 2), match='Av'>
Av
0
2
```

re.match() encontra a equivalência se ela ocorrer no início da string.

resultado.start() e **resultado.end()** retornam a posição **inicial** e **final** do padrão buscado na string considerando intervalo aberto à direita.

Para retornar a string correspondente usamos o método **group()**.

Exemplo

```
import re

resultado = re.search('San', 'Av Santos Dumont')
print (resultado.start())
print (resultado.end())
```

```
<re.Match object; span=(3, 6), match='San'>
3
6
```

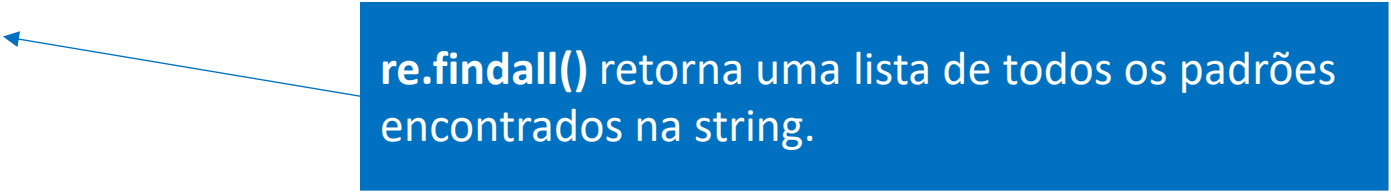
re.search() é similar a **re.match()** retornando a primeira ocorrência encontrada, mas sem se restringir a encontrar equivalência apenas no início da string.

Exemplo

```
import re

resultado = re.findall('n', 'Av Santos Dumont')
print (resultado)
```

```
['n', 'n']
```



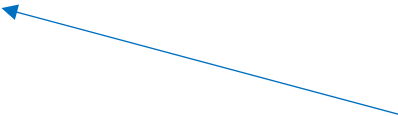
re.findall() retorna uma lista de todos os padrões encontrados na string.

Exemplo

```
import re

resultado = re.finditer('n', 'Av Santos Dumont')
for correspondencia in resultado:
    print (correspondencia.group(0))
    print (correspondencia.start())
    print (correspondencia.end())
```

```
n
5
6
n
14
15
```



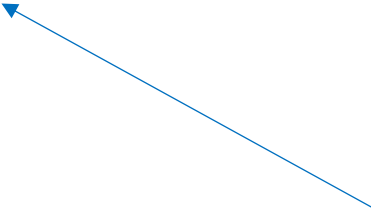
re.finditer() retorna um iterador sobre todas as correspondências não sobrepostas para o padrão na string.

Exemplo

```
import re

resultado = re.split('o', 'Av Santos Dumont')
print (resultado)
```

```
['Av Sant', 's Dum', 'nt']
```



re.split() divide a string pelas ocorrências do padrão fornecido retornando uma lista.

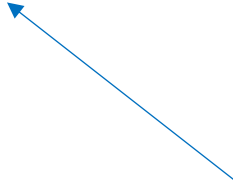
Se o padrão não apresentar ocorrências é retornada uma lista com um único elemento correspondente à toda string.

Exemplo

```
import re
```

```
resultado = re.sub('Av', 'Avenida', 'Av Santos Dumont')  
print (resultado)
```

Avenida Santos Dumont



re.sub() busca um padrão de string e o substitui por uma nova substring.

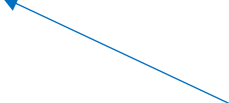
Exemplo

- Retornar todas as palavras de uma string que se iniciam com vogais.

```
import re
```

```
resultado = re.findall(r'\b[aeiouAEIOU]\w+', 'Alberto Santos Dumont foi o primeiro a  
decolar a bordo de um avião impulsionado por um motor a gasolina.')  
print (resultado)
```

```
['Alberto', 'um', 'avião', 'impulsionado', 'um']
```



A expressão regular **[aeiouAEIOU]** em conjunto com **\b** garante que o primeiro caractere de cada palavra seja uma das opções da lista, ou seja, **aeiouAEIOU**.

O **\w** corresponde a um caractere alfanumérico, incluindo sublinhado, sendo equivalente a **[A-Za-z0-9_]**.

O **+** faz com que a expressão regular resultante corresponda a 1 ou mais repetições da expressão regular anterior, assim, **ab+** irá corresponder a 'a' seguido por qualquer número diferente de zero de 'b's e não corresponderá apenas a 'a'.

O **r** no começo da string padrão é usado para designar ao Python uma *string raw* (a string será interpretada como uma string literal).

Exemplo

- Retornar todas as palavras de uma string que se iniciam com vogais.

```
import re
```

```
resultado = re.findall(r'\b[aeiouAEIOU]\w\w+', 'Alberto Santos Dumont foi o primeiro  
a decolar a bordo de um avião impulsionado por um motor a gasolina.')
```

```
print (resultado)
```

```
['Alberto', 'avião', 'impulsionado']
```


Funções lambda

- Também conhecidas como funções anônimas, são funções que não têm um nome e que podem ser definidas em uma única linha de código.
- São comumente usadas como uma forma de criar funções simples que podem ser usadas como argumentos para funções de ordem superior ou em situações em que uma função somente é necessária uma única vez.

`lambda` argumentos: expressão

- Onde:
 - a palavra-chave `lambda` é usada para criar a função `lambda`.
 - os argumentos são os parâmetros de entrada da função, separados por vírgula.
 - a função `lambda` retorna o resultado da expressão, que é o código que será executado quando a função for chamada.

```
operacao = lambda x, y: 0.3 * (x + y)
resultado = operacao(3, 4)
print(resultado)
```

Funções lambda

- As funções lambda podem ser usadas como argumentos para funções de ordem superior como **map()**.
- A função **map()** permite executar a mesma operação em todos os elementos de um iterável de entrada (uma lista, um conjunto, uma tupla) gerando um novo iterável.

```
map(funcao, iteravel1, iteravel2, ..., iteravelN)
```

```
listaX = [3, 5, 6]
listaY = [4, 6, 8]
operacao = lambda x, y: 0.3 * (x + y)
resultado = list(map(operacao, listaX, listaY))
print(resultado)
```

```
lista = [[3, 4], [5, 6], [6, 8]]
operacao = lambda x: 0.3 * (x[0] + x[1])
resultado = list(map(operacao, lista))
print(resultado)
```

Funções lambda

- As funções lambda podem ser usadas como argumentos para funções de ordem superior como **filter()**.
- A função **filter()** é utilizada para filtrar elementos de iteráveis que satisfaçam determinadas condições.

```
filter(funcao, iteravel)
```

- Onde:
 - **função** é a função de filtragem dos elementos, que deve retornar **True** ou **False**.
 - **iteravel** é a estrutura na qual a função será aplicada.

```
def par(numero):  
    return (numero % 2 == 0)  
  
lista = range(10)  
print(list(filter(lambda x: x % 2 == 0, lista)))  
  
lista = range(10)  
print(list(filter(par, lista)))
```

Função reduce

- Aplica uma função de dois parâmetros cumulativamente aos elementos de um iterável, começando com um parâmetro inicial opcional, de modo a reduzir os elementos do iterável a um único valor.
- Se a parâmetro inicial receber algum valor ele será colocado no cálculo antes dos elementos do iterável, além de servir como valor padrão quando o iterável estiver vazio.
- Para usá-la é necessário importar o pacote **functools**.

```
reduce(funcao, iterável [, valor inicial])
```

- Onde:
 - **função** que será aplica aos elementos do objeto iterável.
 - **iteravel** é a estrutura na qual a função será aplicada.

Função reduce

```
from functools import reduce

def somar(x, y):
    return (x + y)

numeros = [1, 2, 3, 4, 5]
valor = reduce(somar, numeros)

print ('O valor da soma é %d.' % (valor))
```

```
from functools import reduce

def acharMaximo(x, y):
    return x if (x > y) else y

numeros = [-1, 10, 2, 30, 0, 23]
valor = reduce(acharMaximo, numeros)

print ('O maior valor é %d.' % (valor))
```

Função zip

- Permite iterar duas ou mais listas ao mesmo tempo, retornando uma lista de tuplas.

```
zip(iteravel1, iteravel2, ..., iteravelN)
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [0, 7, 3, 5, 8]
```

```
print(list(zip(x, y)))
```

Função enumerate

- Retornar o índice de cada item de uma sequência recebida como parâmetro, retornando uma lista de tuplas.

```
enumerate(iteravel)
```

```
frutas = ['banana', 'uva', 'abacaxi', 'limão', 'melancia']
```

```
print(list(enumerate(frutas)))
```

```
for indice, valor in enumerate(frutas):  
    print(indice, valor)
```

Compreensão de lista

- Compreensão de lista (*list comprehension*) É uma construção sintática disponível em algumas linguagens de programação para criação de uma lista baseada em listas existentes. Ela segue a notação de definição de conjuntos.

$$S = \{x^2 \mid x \in N, x > 2\}$$

- Sendo:
 - N , é o conjunto de entrada.
 - x , é a variável que representa os elementos do conjunto de entrada.
 - $x > 2$, é uma função predicado que atua como filtro nos elementos do conjunto de entrada.
 - x^2 , a função de saída.

```
novaLista = [expressão for item in iterável if condição]
```

```
S = [x**2 for x in range(10) if x > 2]
```


Compreensão de lista

- É uma maneira elegante e concisa de criar e executar tarefas sobre listas.
- Em Python, ela permite gerar uma nova lista aplicando uma expressão a cada item de uma sequência ou iterável (como listas, strings, ranges), muitas vezes com uma condição opcional.

```
lista = ["par" if x % 2 == 0 else "ímpar" for x in range(5)]
```

```
matriz = [[i * j for j in range(4)] for i in range(4)]
```

- Observações:
 - Código mais conciso e legível, quando bem usado.
 - Frequentemente mais rápido que um **for** tradicional.
 - Compreensões muito complexas (muitos **if**, **else**, ou aninhamentos), podem prejudicar a legibilidade.

Expressões geradoras

- São semelhantes às **compreensão de listas**, mas com **parênteses** em vez de **colchetes**.

```
f = (x**2 for x in range(10))
```

- O resultado é um **objeto gerador** que sabe como fazer **iterações** por uma **sequência de valores**.
- Ao contrário de uma compreensão de listas, ele **não calcula todos os valores de uma vez**; ele **espera pelo pedido** através da função integrada **next()** que retorna o próximo valor do gerador.

```
next(f)
```

- Quando o final da sequência é atingido, **next()** cria uma exceção **StopIteration**.

Expressões geradoras

- É possível usar um loop **for** para fazer a iteração pelos valores.

```
f = (x**2 for x in range(10))
```

```
for valor in f:  
    print(valor)
```

- O objeto gerador **monitora a posição** em que está na sequência, portanto o loop for continua de onde **next()** parou, assim, uma vez que o gerador se esgotar, ele continua criando **StopException**.
- Muitas vezes são usadas com funções como **sum()**, **max()** e **min()**.

```
sum(x**2 for x in range(10))
```

Expressões geradoras x compreensão de listas

- Se o uso da memória for um problema, então as expressões geradoras são preferíveis.
- O uso de listas é preferível caso, por exemplo, seja necessário iterar mais de uma vez sobre os elementos.
- Um gerador produz um valor e depois se desfaz dele, então se for preciso acessar esse valor novamente, seria necessário computá-lo mais uma vez.
- As listas permitem recuperar um elemento pelo índice, ou até mesmo recuperar sublistas, dentre outros métodos exclusivos de uma lista.
- Listas são sempre finitas, pois todos os seus valores são calculados de uma vez, o gerador, por outro lado, produz um valor por vez e assim podemos ter um gerador com a capacidade infinita.
- Para uma simples iteração não haverá muita diferença entre um e outro, escolha o que preferir e, caso encontre algum problema como memória ou falta de métodos, por exemplo, considere realizar a troca.

Funções geradoras

- Geradores são uma classe especial de funções que simplificam a tarefa de escrever iteradores.
- Funções regulares calculam um valor e o retornam, mas os geradores retornam um iterador que retorna um fluxo de valores.
- Em vez da palavra chave **return**, funções geradoras usam o **yield**, que indica o retorno do **next()** do gerador criado.
- A função **next()** executa o código da **função geradora** até encontrar um **yield** e quando este é encontrado, ela retorna o valor passado para ele e pausa a execução da função.

Funções geradoras

- Geradores são uma classe especial de funções que simplificam a tarefa de escrever iteradores.
- Funções regulares calculam um valor e o retornam, mas os geradores retornam um iterador que retorna um fluxo de valores.
- Em vez da palavra chave **return**, funções geradoras usam o **yield**, que indica o retorno do **next()** do gerador criado.
- A função **next()** executa o código da **função geradora** até encontrar um **yield** e quando este é encontrado, ela retorna o valor passado para ele e pausa a execução da função.

```
def criarGerador():  
    numero = 2  
    yield numero  
    numero = numero ** 2  
    yield numero  
    numero = numero ** 2  
    yield numero
```

```
gerador = criarGerador()  
next(gerador)
```

```
def criarGeradorQuadrados(n):  
    for i in range(n):  
        yield i ** 2
```

```
gerador = criarGeradorQuadrados(5)  
next(gerador)
```

Any e all

- O Python tem uma função integrada, **any()**, que recebe uma sequência de valores booleanos e retorna **True** se algum dos valores for **True**.

```
lista = [False, True, False]
print(any(lista))

saldo = [200, 100, 23, -10, 18, 0]
if (any(x < 0 for x in saldo)):
    print('Existe saldo negativo.')
else:
    print('Não existe saldo negativo.')
```

- Se o objeto iterável estiver vazio, a função **any()** retornará **False**.
- A função integrada **all()** retorna **True** se todos os elementos forem verdadeiros.

```
lista = [False, True, False]
print(all(lista))

saldo = [200, 100, 23, -10, 18, 0]
if (all(x < 0 for x in saldo)):
    print('Todos os saldos são negativos.')
else:
    print('Existe pelo menos um saldo não negativo.')
```

Alguns tipos de dados

list

Sequência ou coleção de dados homogêneos ou heterogêneos.

Os elementos de uma lista são delimitados por colchetes `[]` e separados por vírgulas.

São mutáveis, ou seja, a qualquer momento, um item pode ser incluído, removido e alterado.

tupla

Sequências imutáveis (não permite a adição ou remoção de elementos), tipicamente usadas para armazenar coleções de dados heterogêneos.

Sequência de valores separados por vírgulas e envolvidos por parênteses `()`.

É possível criar tuplas que contenham objetos mutáveis, como listas.

set (conjuntos)

Coleção desordenada e mutável de elementos, sem elementos repetidos.

Suportam operações matemáticas como união, interseção, diferença e diferença simétrica.

Chaves `{ }` ou a função `set()` podem ser usados para criar conjuntos.

dict (dicionários)

Coleção não-ordenada de pares chave:valor, onde as chaves são únicas em uma dada instância do dicionário.

Os itens do dicionário são ordenáveis, alteráveis e não permitem duplicatas.

Dicionários são delimitados por chaves `{ }` e contém uma sequência de pares chave:valor separadas por vírgulas.

Listas

- É uma **sequência** ou coleção de **dados homogêneos ou heterogêneos**.
- Os **valores de uma lista**, também chamados de **elementos** ou **itens**, podem ser de **tipos diferentes** e até mesmo **outras listas (sublistas)**, sendo delimitados por colchetes `[]` e separados por vírgulas.

```
indice = [3.99, -2.45, 1.23, 2.34, 0.89, 3.57, -1.07]
acao = ['AZUL4', 'AMER3', 'BBAS3', 'BBDC4']
variacao = [['ABEV3', 1.23], ['CRFB3', 2.03], ['BRKM5', 0.35]]
carteira = ['Pedro Sanchez', 900.00, [['ABEV3', 25], ['AZUL4', 75]]]

print (indice) # lista inteira
print (acao[0]) # primeiro item da lista
print (variacao[1]) # segundo item da lista
print (variacao[1][0]) # na segunda sublista o primeiro item
print (carteira[2]) # terceiro item
print (carteira[2][1]) # no terceiro item a segunda sublista
print (carteira[2][1][1]) # no terceiro item, na segunda sublista, o segundo item
```

Listas

```
indice = [3.99, -2.45, 1.23, 2.34, 0.89, 3.57, -1.07]
acao = ['AZUL4', 'AMER3', 'BBAS3', 'BBDC4']
variacao = [['ABEV3', 1.23], ['CRFB3', 2.03], ['BRKM5', 0.35]]
carteira = ['Pedro Sanchez', 900.00, [['ABEV3', 25], ['AZUL4', 75]]]

print (indice) # lista inteira
print (acao[0]) # primeiro item da lista
print (variacao[1]) # segundo item da lista
print (variacao[1][0]) # na segunda sublista o primeiro item
print (carteira[2]) # terceiro item
print (carteira[2][1]) # no terceiro item a segunda sublista
print (carteira[2][1][1]) # no terceiro item, na segunda sublista, o segundo item
```

Listas

```
indice = [3.99, -2.45, 1.23, 2.34, 0.89, 3.57, -1.07]
acao = ['AZUL4', 'AMER3', 'BBAS3', 'BBDC4']
variacao = [['ABEV3', 1.23], ['CRFB3', 2.03], ['BRKM5', 0.35]]
carteira = ['Pedro Sanchez', 900.00, [['ABEV3', 25], ['AZUL4', 75]]]

print (indice) # lista inteira
print (acao[0]) # primeiro item da lista
print (variacao[1]) # segundo item da lista
print (variacao[1][0]) # na segunda sublista o primeiro item
print (carteira[2]) # terceiro item
print (carteira[2][1]) # no terceiro item a segunda sublista
print (carteira[2][1][1]) # no terceiro item, na segunda sublista, o segundo item
```

Listas

```
indice = [3.99, -2.45, 1.23, 2.34, 0.89, 3.57, -1.07]
acao = ['AZUL4', 'AMER3', 'BBAS3', 'BBDC4']
variacao = [['ABEV3', 1.23], ['CRFB3', 2.03], ['BRKM5', 0.35]]
carteira = ['Pedro Sanchez', 900.00, [['ABEV3', 25], ['AZUL4', 75]]]

print (indice) # lista inteira
print (acao[0]) # primeiro item da lista
print (variacao[1]) # segundo item da lista
print (variacao[1][0]) # na segunda sublista o primeiro item
print (carteira[2]) # terceiro item
print (carteira[2][1]) # no terceiro item a segunda sublista
print (carteira[2][1][1]) # no terceiro item, na segunda sublista, o segundo item
```

Listas

```
indice = [3.99, -2.45, 1.23, 2.34, 0.89, 3.57, -1.07]
acao = ['AZUL4', 'AMER3', 'BBAS3', 'BBDC4']
variacao = [['ABEV3', 1.23], ['CRFB3', 2.03], ['BRKM5', 0.35]]
carteira = ['Pedro Sanchez', 900.00, [['ABEV3', 25], ['AZUL4', 75]]]

print (indice) # lista inteira
print (acao[0]) # primeiro item da lista
print (variacao[1]) # segundo item da lista
print (variacao[1][0]) # na segunda sublista o primeiro item
print (carteira[2]) # terceiro item
print (carteira[2][1]) # no terceiro item a segunda sublista
print (carteira[2][1][1]) # no terceiro item, na segunda sublista, o segundo item
```

Listas

```
indice = [3.99, -2.45, 1.23, 2.34, 0.89, 3.57, -1.07]
acao = ['AZUL4', 'AMER3', 'BBAS3', 'BBDC4']
variacao = [['ABEV3', 1.23], ['CRFB3', 2.03], ['BRKM5', 0.35]]
carteira = ['Pedro Sanchez', 900.00, [['ABEV3', 25], ['AZUL4', 75]]]

print (indice) # lista inteira
print (acao[0]) # primeiro item da lista
print (variacao[1]) # segundo item da lista
print (variacao[1][0]) # na segunda sublista o primeiro item
print (carteira[2]) # terceiro item
print (carteira[2][1]) # no terceiro item a segunda sublista
print (carteira[2][1][1]) # no terceiro item, na segunda sublista, o segundo item
```

Listas

```
indice = [3.99, -2.45, 1.23, 2.34, 0.89, 3.57, -1.07]
acao = ['AZUL4', 'AMER3', 'BBAS3', 'BBDC4']
variacao = [['ABEV3', 1.23], ['CRFB3', 2.03], ['BRKM5', 0.35]]
carteira = ['Pedro Sanchez', 900.00, [['ABEV3', 25], ['AZUL4', 75]]]

print (indice) # lista inteira
print (acao[0]) # primeiro item da lista
print (variacao[1]) # segundo item da lista
print (variacao[1][0]) # na segunda sublista o primeiro item
print (carteira[2]) # terceiro item
print (carteira[2][1]) # no terceiro item a segunda sublista
print (carteira[2][1][1]) # no terceiro item, na segunda sublista, o segundo item
```

Listas

```
indice = [3.99, -2.45, 1.23, 2.34, 0.89, 3.57, -1.07]
acao = ['AZUL4', 'AMER3', 'BBAS3', 'BBDC4']
variacao = [['ABEV3', 1.23], ['CRFB3', 2.03], ['BRKM5', 0.35]]
carteira = ['Pedro Sanchez', 900.00, [['ABEV3', 25], ['AZUL4', 75]]]

print (indice) # lista inteira
print (acao[0]) # primeiro item da lista
print (variacao[1]) # segundo item da lista
print (variacao[1][0]) # na segunda sublista o primeiro item
print (carteira[2]) # terceiro item
print (carteira[2][1]) # no terceiro item a segunda sublista
print (carteira[2][1][1]) # no terceiro item, na segunda sublista, o segundo item
```


Listas

- Para definir uma lista como **vazia** ou **limpar** uma lista fazemos **nomeVariavelLista = []**.
- A função **len** retorna o **número de elementos** de uma lista, sendo que uma sublista é contada como um elemento da lista que a contém.
- **Índices fora do intervalo provocam um erro de execução**. Lembrando que o primeiro índice é **0** e o último índice é o número de elementos da lista **menos 1**.
- Para percorrer uma lista podemos usar o comando **for**.

```
for elemento in acao:  
    print(elemento)
```

Percorre toda a lista **acao** elemento por elemento.

```
for i in range(len(acao)):  
    print(acao[i])
```

Percorre toda a lista **acao** usando o índice da lista. A função range gera uma sequência de **0** ao tamanho da lista **menos 1**.

Listas

- Assim como as strings, **listas podem ser fatiadas**.
- O **fatiamento** de uma lista **cria uma nova lista** selecionando um intervalo (fatia) da lista da posição **a** (inclusive) até a posição **b** (exclusive) de **n** em **n**.

`lista[a:b:n]`

- Possibilidades de fatiamento:
 - `lista[a:b]` - cria uma cópia de **a** (inclusive) até **b** (exclusive).
 - `lista[a:]` - cria uma cópia a partir de **a** (inclusive).
 - `lista[:b]` - cria uma cópia até **b** (exclusive).
 - `lista[:]` - cria uma cópia de todos os elementos.
 - `lista[a:b:n]` - cria uma cópia de **a** (inclusive) até **b** (exclusive) de **n** em **n** elementos

Listas

- **São mutáveis**, ou seja, a qualquer momento, um item pode ser **incluído**, **removido** e **alterado**.
- Alguns métodos disponíveis para listas.

Método	Descrição
<code>append(x)</code>	Insere um elemento ao final da lista.
<code>insert(i, x)</code>	Insere um elemento em uma posição específica na lista.
<code>remove(x)</code>	Remove um elemento da lista.
<code>del lista[a:b]</code>	Remove os elementos de índice a até b menos 1.
<code>pop(i)</code>	Remove e retorna o elemento de índice i da lista.
<code>index(x)</code>	Busca um elemento na lista e retorna seu índice.
<code>sort()</code>	Ordena a lista (os itens devem ser do mesmo tipo).

Exemplo

- Elabore uma função chamada **intersecao** que receba duas listas X e Y contendo valores numéricos e retorne a interseção entre elas.

Exemplo

- Elabore uma função chamada **intersecao** que receba duas listas X e Y contendo valores numéricos e retorne a interseção entre elas.

```
def intersecao (lista1, lista2):  
    listaIntersecao = [] # Inicializa a lista de retorno como vazia.  
    for elemento in lista1: # Percorre a lista1 elemento por elemento.  
        if (elemento in lista2): # Verifica (in) se o elemento está na lista2.  
            listaIntersecao.append(elemento) # Insere ao final da lista.  
    return (listaIntersecao)  
  
X = [1.2, 2, 3.9, 4, 5] # Cria a lista X.  
Y = [1, 3.9, 5, 4.2, 6, 15] # Cria a lista Y.  
print (intersecao (X, Y)) # Chama a função e imprime a lista retornada.
```

Exercício

- Sabendo que a função **abs** retorna o valor absoluto da diferença entre dois números, elabore uma função que receba uma lista contendo apenas números e retorne o valor mais próximo da média dos elementos.

Exercício

```
def calculaMedia(lista): # Função para calcular a média da lista.
    soma = 0
    for elemento in lista: # Percorre os elementos.
        soma += elemento
    media = soma / len(lista)
    return (media)

def valorMaisProximo(lista):
    media = calculaMedia(lista) # Calcula a média.
    maisProximo = lista[0] # Seleciona o primeiro elemento da lista.
    maisProximoDiferenca = abs(maisProximo - media) # Calcula a diferença.
    for i in range(1, len(lista)): # Percorre a lista a partir do segundo elemento.
        diferenca = abs(lista[i] - media) # Calcula a diferença.
        if (diferenca < maisProximoDiferenca): # Verifica se a diferença é menor.
            maisProximoDiferenca = diferenca # Guarda a nova diferença.
            maisProximo = lista[i] # Guarda o novo elemento de menor diferença.

    return (maisProximo)

lista = [6] # Executa a função para uma lista com um único elemento.
print(valorMaisProximo(lista))
lista = [-1, 2, 5, 3.5] # Executa a função para uma lista com cinco elementos.
print(valorMaisProximo(lista))
```

O método **mean()** do pacote **statistics** é uma alternativa mais simples.

Tuplas

- Consistem em uma **sequência** de valores **separados por vírgulas** e **envolvidos por parênteses**.
- Enquanto as **listas** são **sequências mutáveis**, normalmente usadas para **armazenar coleções de itens homogêneos**, **tuplas** são **sequências imutáveis**, tipicamente **usadas para armazenar coleções de dados heterogêneos**.
- **É possível criar tuplas que contenham objetos mutáveis, como listas.**

```
"""
```

```
Tupla que armazena o código do cliente dentro de uma seguradora e uma  
lista contendo número de chassi e ano de compra de seus veículos.
```

```
"""
```

```
t = (768, [['9BGRD08X04G117974', 2017], ['9BHTG97Z73J236723', 2018]])
```

```
print(t[0]) # Primeiro elemento.
```

```
print(t) # Tupla inteira.
```

```
t[1][0][1] = 2016 # Alterando 2017 para 2016 (o elemento 1 de t é uma lista).
```

```
print(t)
```

```
t[0] = 123 # Tentando mudar o código do cliente.
```


Tuplas

```
"""
    Tupla que armazena o código do cliente dentro de uma seguradora e uma
    lista contendo número de chassi e ano de compra de seus veículos.
"""
t = (768, [['9BGRD08X04G117974', 2017], ['9BHTG97Z73J236723', 2018]])

print(t[0]) # Primeiro elemento.

print(t) # Tupla inteira.

t[1][0][1] = 2016 # Alterando 2017 para 2016 (o elemento 1 de t é uma lista).
print(t)

t[0] = 123 # Tentando mudar o código do cliente.
```

Tuplas

```
"""
    Tupla que armazena o código do cliente dentro de uma seguradora e uma
    lista contendo número de chassi e ano de compra de seus veículos.
"""
t = (768, [['9BGRD08X04G117974', 2017], ['9BHTG97Z73J236723', 2018]])

print(t[0]) # Primeiro elemento.

print(t) # Tupla inteira.

t[1][0][1] = 2016 # Alterando 2017 para 2016 (o elemento 1 de t é uma lista).
print(t)

t[0] = 123 # Tentando mudar o código do cliente.
```

Tuplas

```
"""
```

```
Tupla que armazena o código do cliente dentro de uma seguradora e uma  
lista contendo número de chassi e ano de compra de seus veículos.
```

```
"""
```

```
t = (768, [['9BGRD08X04G117974', 2017], ['9BHTG97Z73J236723', 2018]])
```

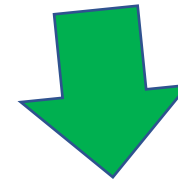
```
print(t[0]) # Primeiro elemento.
```

```
print(t) # Tupla inteira.
```

```
t[1][0][1] = 2016 # Alterando 2017 para 2016 (o elemento 1 de t é uma lista).
```

```
print(t)
```

```
t[0] = 123 # Tentando mudar o código do cliente.
```



```
(768, [['9BGRD08X04G117974', 2016], ['9BHTG97Z73J236723', 2018]])
```

Tuplas

```
"""
    Tupla que armazena o código do cliente dentro de uma seguradora e uma
    lista contendo número de chassi e ano de compra de seus veículos.
"""
t = (768, [['9BGRD08X04G117974', 2017], ['9BHTG97Z73J236723', 2018]])

print(t[0]) # Primeiro elemento.

print(t) # Tupla inteira.

t[1][0][1] = 2016 # Alterando 2017 para 2016 (o elemento 1 de t é uma lista).
print(t)

t[0] = 123 # Tentando mudar o código do cliente.
```



Observe que a **lista de veículos do cliente** pode e deve ser alterada, mas seu **código de cliente**, que é o seu **identificador** dentro da seguradora, não deve ser alterado.

TypeError: 'tuple' object does not support item assignment

Tuplas

- Apesar de **tuplas** serem **similares a listas**, elas são frequentemente utilizadas em **situações diferentes** e com **propósitos distintos**.
- Uma **lista** pode ter **elementos adicionados** ou **removidos** a qualquer momento, enquanto uma **tupla**, uma vez definida, **não permite a adição** ou **remoção de elementos**.
- Sua característica de **imutabilidade** oferece segurança nas informações armazenadas e, por isso, uma das **finalidades da tupla** é **armazenar uma sequência de dados que não será modificada em outras partes do código**.

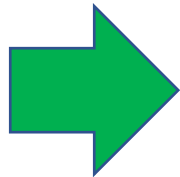
Tuplas

- Seus elementos são **acessados via índice** (visto anteriormente) ou **desempacotamento**.

```
t = (768, [['9BGRD08X04G117974', 2017], ['9BHTG97Z73J236723', 2018]])
```

```
codigo, listaVeiculo = t # Desempacotamento
```

```
print(codigo)
```



768

```
print(listaVeiculo)
```



```
 [['9BGRD08X04G117974', 2017], ['9BHTG97Z73J236723', 2018]]
```

No desempacotamento, a variável **codigo** irá receber o **código do cliente** e a variável **listaVeiculo** irá receber a **lista de veículos do cliente**.

Tuplas

- Uma vez criada, a **tupla não pode ter seus elementos reordenados**.
- A função **len** retorna o **número de elementos** de uma tupla.
- O operador **in** permite verificar se um **elemento existe em uma tupla**, retornando **True** ou **False** de acordo com o resultado da pesquisa.
- **Índices fora do intervalo provocam um erro de execução**. Lembrando que o primeiro índice é **0** e o último índice é o número de elementos da tupla **menos 1**.
- Para percorrer uma tupla podemos usar o comando **for**.

```
for elemento in t:  
    print(elemento)
```

Percorre toda a tupla **t** elemento por elemento.

```
for i in range((len(t))):  
    print(t[i])
```

Percorre toda a tupla **t** usando o **índice da tupla**. A função **range** gera uma sequência de **0** ao tamanho da tupla **menos 1**.

Tuplas

- Assim como as listas, **tuplas podem ser fatiadas**.
- O fatiamento de uma tupla cria uma nova tupla selecionando um intervalo (fatia) da tupla da posição **a** (inclusive) até a posição **b** (exclusive) de **n** em **n**.

tupla[**a:b:n**]

- Possibilidades de fatiamento:
 - tupla[a:b] - cria uma cópia de **a** (inclusive) até **b** (exclusive).
 - tupla[a:] - cria uma cópia a partir de **a** (inclusive).
 - tupla[:b] - cria uma cópia até **b** (exclusive).
 - tupla[:] - cria uma cópia de todos os elementos.
 - tupla[a:b:n] - cria uma cópia de **a** (inclusive) até **b** (exclusive) de **n** em **n** elementos.

Conjuntos

- Um **conjunto** é uma **coleção desordenada** de elementos, **sem elementos repetidos**.
- Suportam operações matemáticas como **união**, **interseção**, **diferença** e **diferença simétrica** e, por isso, são comumente **usados na verificação eficiente da existência de objetos e na eliminação de itens duplicados**.
- **Chaves** ou a função **set()** podem ser usados para criar conjuntos, sendo que para criar um **conjunto vazio** deve ser usado **set()** e não **{}**.

```
A = {'Dolar', 'Euro', 'Real', 'Libra', 'Real', 'Dolar', 'Peso'}
B = {'Euro', 'Libra', 'Franco'}
print(A) # Sem repetição.
print('Dolar' in A)
print('Rupia' in A)
print(A - B) # Moeda em A mas não em B.
print(A | B) # Moeda em A ou em B ou em ambos.
print(A & B) # Moeda em A e em B.
print(A ^ B) # Moeda em A ou em B mas não em ambos.
```

Conjuntos

```
A = {'Dolar', 'Euro', 'Real', 'Libra', 'Real', 'Dolar', 'Peso'}
```

```
B = {'Euro', 'Libra', 'Franco'}
```

```
print(A) # Sem repetição.
```

```
print('Dolar' in A)
```

```
print('Rupia' in A)
```

```
print(A - B) # Moeda em A mas não em B.
```

```
print(A | B) # Moeda em A ou em B ou em ambos.
```

```
print(A & B) # Moeda em A e em B.
```

```
print(A ^ B) # Moeda em A ou em B mas não em ambos.
```

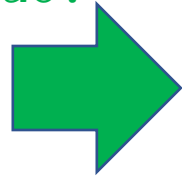
Conjuntos

```
A = {'Dolar', 'Euro', 'Real', 'Libra', 'Real', 'Dolar', 'Peso'}
```

```
B = {'Euro', 'Libra', 'Franco'}
```

```
print(A) # Sem repetição.
```

```
print('Dolar' in A)
```



True

```
print('Rupia' in A)
```

```
print(A - B) # Moeda em A mas não em B.
```

```
print(A | B) # Moeda em A ou em B ou em ambos.
```

```
print(A & B) # Moeda em A e em B.
```

```
print(A ^ B) # Moeda em A ou em B mas não em ambos.
```

Conjuntos

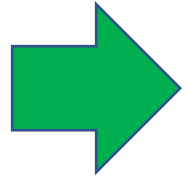
```
A = {'Dolar', 'Euro', 'Real', 'Libra', 'Real', 'Dolar', 'Peso'}
```

```
B = {'Euro', 'Libra', 'Franco'}
```

```
print(A) # Sem repetição.
```

```
print('Dolar' in A)
```

```
print('Rupia' in A)
```



False

```
print(A - B) # Moeda em A mas não em B.
```

```
print(A | B) # Moeda em A ou em B ou em ambos.
```

```
print(A & B) # Moeda em A e em B.
```

```
print(A ^ B) # Moeda em A ou em B mas não em ambos.
```

Conjuntos

```
A = {'Dolar', 'Euro', 'Real', 'Libra', 'Real', 'Dolar', 'Peso'}
```

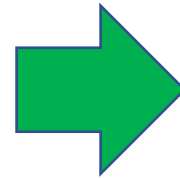
```
B = {'Euro', 'Libra', 'Franco'}
```

```
print(A) # Sem repetição.
```

```
print('Dolar' in A)
```

```
print('Rupia' in A)
```

```
print(A - B) # Moeda em A mas não em B.
```



```
{'Real', 'Peso', 'Dolar'}
```

```
print(A | B) # Moeda em A ou em B ou em ambos.
```

```
print(A & B) # Moeda em A e em B.
```

```
print(A ^ B) # Moeda em A ou em B mas não em ambos.
```

Conjuntos

```
A = {'Dolar', 'Euro', 'Real', 'Libra', 'Real', 'Dolar', 'Peso'}
```

```
B = {'Euro', 'Libra', 'Franco'}
```

```
print(A) # Sem repetição.
```

```
print('Dolar' in A)
```

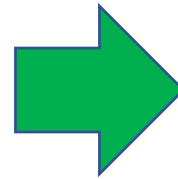
```
print('Rupia' in A)
```

```
print(A - B) # Moeda em A mas não em B.
```

```
print(A | B) # Moeda em A ou em B ou em ambos.
```

```
print(A & B) # Moeda em A e em B.
```

```
print(A ^ B) # Moeda em A ou em B mas não em ambos.
```



```
{'Real', 'Franco', 'Libra',  
'Peso', 'Dolar', 'Euro'}
```

Conjuntos

```
A = {'Dolar', 'Euro', 'Real', 'Libra', 'Real', 'Dolar', 'Peso'}
```

```
B = {'Euro', 'Libra', 'Franco'}
```

```
print(A) # Sem repetição.
```

```
print('Dolar' in A)
```

```
print('Rupia' in A)
```

```
print(A - B) # Moeda em A mas não em B.
```

```
print(A | B) # Moeda em A ou em B ou em ambos.
```

```
print(A & B) # Moeda em A e em B.
```



```
{'Euro', 'Libra'}
```

```
print(A ^ B) # Moeda em A ou em B mas não em ambos.
```

Conjuntos

```
A = {'Dolar', 'Euro', 'Real', 'Libra', 'Real', 'Dolar', 'Peso'}
```

```
B = {'Euro', 'Libra', 'Franco'}
```

```
print(A) # Sem repetição.
```

```
print('Dolar' in A)
```

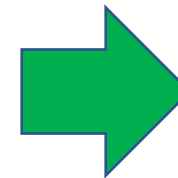
```
print('Rupia' in A)
```

```
print(A - B) # Moeda em A mas não em B.
```

```
print(A | B) # Moeda em A ou em B ou em ambos.
```

```
print(A & B) # Moeda em A e em B.
```

```
print(A ^ B) # Moeda em A ou em B mas não em ambos.
```



```
{'Franco', 'Real',  
'Peso', 'Dolar'}
```


Conjuntos

- Alguns métodos disponíveis para conjuntos.

Método	Descrição
<code>s.difference_update(t)</code>	Remove os itens que estão no conjunto t do conjunto s .
<code>s.intersection_update(t)</code>	Faz com que o conjunto s contenha a interseção dele com t .
<code>s.isdisjoin(t)</code>	Retorna True se s e t não tem nenhum item em comum.
<code>s.issubset(t) (s <= t)</code>	Retorna True se s é igual a t ou um subconjunto de t .
<code>s.issuperset(t) (s >= t)</code>	Retorna True se s é igual a t ou t é um subconjunto de s .
<code>s.add(x)</code>	Adiciona ao conjunto s o elemento x .
<code>s.discard(x)</code>	Remove o item x do conjunto s .
<code>s.remove(x)</code>	Remove o item x se ele estiver em s senão retorna um KeyError .
<code>s.update(t)</code>	Adiciona cada item do conjunto t que não está no conjunto s ao conjunto s .

Dicionários

- Coleção **não-ordenada de pares chave:valor**, onde as **chaves** são únicas em uma dada instância do dicionário. As **chaves** podem ser de qualquer tipo **imutável** (como strings e inteiros), sendo **valor** qualquer tipo de dado.
- Os itens do dicionário são **ordenáveis, alteráveis e não permitem duplicatas**.
- Dicionários são delimitados por { }, e contém uma sequência de pares **chave:valor** separadas por vírgulas.

```
# Coleção de clientes representados por código (chave) e nome (valor).
cliente = {345:'Pedro Santos', 675:'Caio Freitas', 879:'Carmem Rios'}
print(cliente) # Todos os clientes.
print(cliente[675]) # Nome do cliente de código 675.
cliente[649] = 'Carlos Lopes' # Incluindo um cliente.
print(cliente)
print(cliente[123]) # Retorna erro de execução pois 123 não existe como chave.
```

Dicionários

```
# Coleção de clientes representados por código (chave) e nome (valor).  
cliente = {345:'Pedro Santos', 675:'Caio Freitas', 879:'Carmem Rios'}
```

```
print(cliente) # Todos os clientes.
```

```
print(cliente[675]) # Nome do cliente de código 675.
```

```
cliente[649] = 'Carlos Lopes' # Incluindo um cliente.  
print(cliente)
```

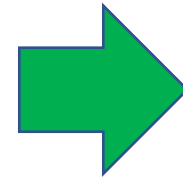
```
print(cliente[123]) # Retorna erro de execução pois 123 não existe como chave.
```

Dicionários

```
# Coleção de clientes representados por código (chave) e nome (valor).  
cliente = {345:'Pedro Santos', 675:'Caio Freitas', 879:'Carmem Rios'}
```

```
print(cliente) # Todos os clientes.
```

```
print(cliente[675]) # Nome do cliente de código 675.
```



Caio Freitas

```
cliente[649] = 'Carlos Lopes' # Incluindo um cliente.  
print(cliente)
```

```
print(cliente[123]) # Retorna erro de execução pois 123 não existe como chave.
```

Dicionários

```
# Coleção de clientes representados por código (chave) e nome (valor).  
cliente = {345:'Pedro Santos', 675:'Caio Freitas', 879:'Carmem Rios'}
```

```
print(cliente) # Todos os clientes.
```

```
print(cliente[675]) # Nome do cliente de código 675.
```

```
cliente[649] = 'Carlos Lopes' # Incluindo um cliente.  
print(cliente)
```



```
{345: 'Pedro Santos', 675: 'Caio Freitas', 879: 'Carmem Rios', 649: 'Carlos Lopes'}
```

```
print(cliente[123]) # Retorna erro de execução pois 123 não existe como chave.
```

Dicionários

```
# Coleção de clientes representados por código (chave) e nome (valor).  
cliente = {345:'Pedro Santos', 675:'Caio Freitas', 879:'Carmem Rios'}
```

```
print(cliente) # Todos os clientes.
```

```
print(cliente[675]) # Nome do cliente de código 675.
```

```
cliente[649] = 'Carlos Lopes' # Incluindo um cliente.  
print(cliente)
```

```
print(cliente[123]) # Retorna erro de execução pois 123 não existe como chave.
```



KeyError: 123

Observe que o dicionário **cliente** não possui a chave **123**.

Dicionários

- Para definir uma dicionário vazio fazemos **nomeVariavel = {}**.
- A função **dict** permite criar dicionários a partir de listas.

```
listaEstado = [('RJ', 'Rio de Janeiro'), ('SP', 'São Paulo')]  
estado = dict(listaEstado)
```

- A função **len** retorna o número de elementos do dicionário.
- Para recuperar um valor do dicionário também pode ser usado o método **get** que recebe o valor da chave para busca.

```
print(estado.get('RJ')) # Retorna 'Rio de Janeiro'  
print(estado.get('ES')) # Retorna None, pois 'ES' não existe como chave.
```

- Para remover um elemento do dicionário pode ser usado o comando **pop**.

```
valor = estado.pop('SP')  
print(valor)
```

- O método **update** permite unir dois dicionários sem que sejam criadas duplicidades de chaves.

Dicionários

- Os operadores **in** e **not in** permitem identificar se um elemento está em um dicionário ou não.

```
print('RJ' in estado)
print('PE' not in estado)
```

- O método **keys** permite obter uma lista das chaves de um dicionário, o método **values** a lista de valores e o método **items** lista de tuplas, onde cada tupla é composta por dois valores: a chave e o valor.

```
print(list(estado.keys()))
print(list(estado.values()))
print(list(estado.items()))
```

O list precisa ser aplicado para realizar a conversão pois os métodos keys retorna um tipo dict_keys e não uma lista.
O list precisa ser aplicado para realizar a conversão pois os métodos values retorna um tipo dict_values e não uma lista.
O list precisa ser aplicado para realizar a conversão pois os métodos items retorna um tipo dict_items e não uma lista.

Dicionários

- Para percorrer um dicionário podemos usar o comando **for**.

```
for chave in estado:  
    print(chave)  
    print(estado[chave])
```

Percorre todo o dicionário **estado** elemento por elemento.

```
for chave, valor in estado.items():  
    print(chave)  
    print(valor)
```

Percorre todo o dicionário **estado** elemento por elemento, sendo que o método **items** permite obter chave e valor simultaneamente.

Exemplo

- Escreva a função chamada **recuperarMensagem**, que recebe uma lista com números inteiros correspondentes a uma mensagem secreta e retornar a mensagem de acordo com a tabela de codificação abaixo:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
' '	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z



Espaço em branco.

Exemplo

```
def recuperarMensagem(mensagemCodificada):  
    tabelaDecodificacao = " abcdefghijklmnopqrstuvwxyz" # Tabela de decodificação.  
    mensagemDecodificada = '' # Mensagem a ser retornada.  
    for codigo in mensagemCodificada: # Percorre os códigos da mensagem codificada passada.  
        mensagemDecodificada = mensagemDecodificada + tabelaDecodificacao[codigo] # Decodificação.  
    return (mensagemDecodificada)  
  
# Corpo do programa.  
  
# Mensagem secreta.  
mensagemSecreta = [20, 5, 14, 8, 1, 0, 21, 13, 1, 0, 2, 15, 1, 0, 20, 1, 18, 4, 5]  
  
# Mensagem original.  
mensagem = recuperarMensagem(mensagemSecreta)  
  
print(mensagem)
```